

Dokumentacja Projektowa: Generowanie Grafów Planarnych

Aleksandra Rybałko, Nella Karczewicz

14 kwietnia 2026

Spis treści

1	Wstęp i Architektura Systemu	2
1.1	Przedmiot Projektu	2
1.2	Modułowa Budowa Programu	2
2	Szczegółowa Analiza Algorytmów	2
2.1	Algorytm Tutte’a (Metoda Barycentryczna)	2
2.2	Algorytm Fruchterman-Reingold (Model Siłowy)	3
2.3	Weryfikacja Planarności i Mechanizm Bezkolizyjny	3
3	Przepływ Danych i Walidacja	4
3.1	Schemat Przetwarzania Danych	4
4	System Diagnostyki (Kody Wyjścia)	5
5	Metodologia Testowania i Scenariusze Weryfikacyjne	5
5.1	Walidacja Składniowa i Odporność modułu I/O	5
5.2	Testy Topologiczne (Logika Grafowa)	6
5.3	Zestawienie Scenariuszy Testowych	6
5.4	Wnioski z procesu testowania	6
6	Podział prac w zespole	7
6.1	Tabela zadań	7
6.2	Opis wykonanych prac	7
7	Podsumowanie	8

1 Wstęp i Architektura Systemu

1.1 Przedmiot Projektu

Projekt "Graph Layout Engine" to specjalistyczne narzędzie konsolowe (CLI), którego zadaniem jest transformacja abstrakcyjnej listy krawędzi grafu w czytelną reprezentację geometryczną 2D. System kładzie szczególny nacisk na poprawność matematyczną (planarność) oraz stabilność działania przy błędnych danych wejściowych.

1.2 Modułowa Budowa Programu

Kod został podzielony na niezależne jednostki logiczne, co zapewnia wysoką czytelność i łatwość konserwacji:

- **graph.h**: Centralny punkt definicji struktur. Zawiera opisy typów `Node`, `Edge` oraz `Graph`, stanowiące wspólny interfejs dla całego systemu.
- **io.c / io.h**: Moduł wejścia/wyjścia. Odpowiada za niskopoziomą obsługę plików, dwufazową walidację składniową, sanityzację danych oraz eksport wyników (TXT/BIN).
- **layout_tutte.c**: Implementacja deterministycznego algorytmu barycentrycznego oraz geometrycznych testów przecięć krawędzi (CCW).
- **fruchterman.c**: Implementacja heurystycznego modelu siłowego wraz z mechanizmami grawitacji i chłodzenia układu.
- **main.c**: Moduł sterujący. Zarządza logiką CLI, koordynuje przepływ danych i realizuje walidację argumentów wywołania.

2 Szczegółowa Analiza Algorytmów

2.1 Algorytm Tutte'a (Metoda Barycentryczna)

Metoda ta stanowi fundament deterministycznego podejścia do wizualizacji grafów planarnych. Opiera się na założeniu, że graf trójspójny narysowany z wypukłą ramą zewnętrzną osiągnie stan bezkolizyjny, jeśli każdy wierzchołek wewnętrzny znajdzie się w środku ciężkości swoich sąsiadów.

Logika i Implementacja: Współrzędne wierzchołków ruchomych v_i są wyznaczone jako średnia arytmetyczna pozycji ich sąsiadów. Proces ten realizowany jest poprzez 200 iteracji relaksacji układu:

$$x_i = \frac{1}{|Adj(v_i)|} \sum_{j \in Adj(v_i)} x_j, \quad y_i = \frac{1}{|Adj(v_i)|} \sum_{j \in Adj(v_i)} y_j$$

Stabilność geometryczną zapewnia tzw. "sztywna rama" – cztery wybrane węzły są blokowane na planie koła, co wymusza wypukłość całego layoutu i zapobiega nakładaniu się krawędzi.

2.2 Algorytm Fruchterman-Reingold (Model Siłowy)

Algorytm ten stanowi realizację heurystyki fizycznej, wykorzystującej analogię układu cząsteczek i sprężyn. W przeciwieństwie do deterministycznej metody Tutte’a, jest to proces stochastyczny, dążący do znalezienia konfiguracji o najniższej energii potencjalnej poprzez symulację oddziaływań międzywęzłowych.

Dynamika Oddziaływań: Fundamentem algorytmu jest balans sił, definiowany przez stałą optymalnej odległości k , zależną od powierzchni obszaru roboczego ($Area$) oraz liczby wierzchołków ($|V|$):

$$k = \sqrt{\frac{Area}{|V|}}$$

W każdej iteracji na wierzchołki działają dwa przeciwstawne rodzaje sił:

- **Siła odpychania (f_r):** Obliczana dla każdej pary węzłów, zapobiega ich nadmiernemu skupianiu: $f_r(d) = \frac{k^2}{d}$.
- **Siła przyciągania (f_a):** Działająca wyłącznie wzdłuż zdefiniowanych krawędzi, dążąca do skrócenia połączeń: $f_a(d) = \frac{d^2}{k}$.

Optymalizacja i Simulated Annealing: Stabilizację układu zapewnia mechanizm „chłodzenia” (*Simulated Annealing*). Wprowadza on parametr temperatury t , który ogranicza maksymalne przesunięcie wierzchołków w jednym kroku. Temperatura maleje geometrycznie w każdej iteracji:

$$t_{new} = t_{old} \cdot 0.95$$

Proces ten pozwala na swobodną eksplorację przestrzeni w początkowej fazie (wysoka temperatura) oraz precyzyjne dociągnięcie układu do punktu równowagi w fazie końcowej (niska temperatura).

2.3 Weryfikacja Planarności i Mechanizm Bezkolizyjny

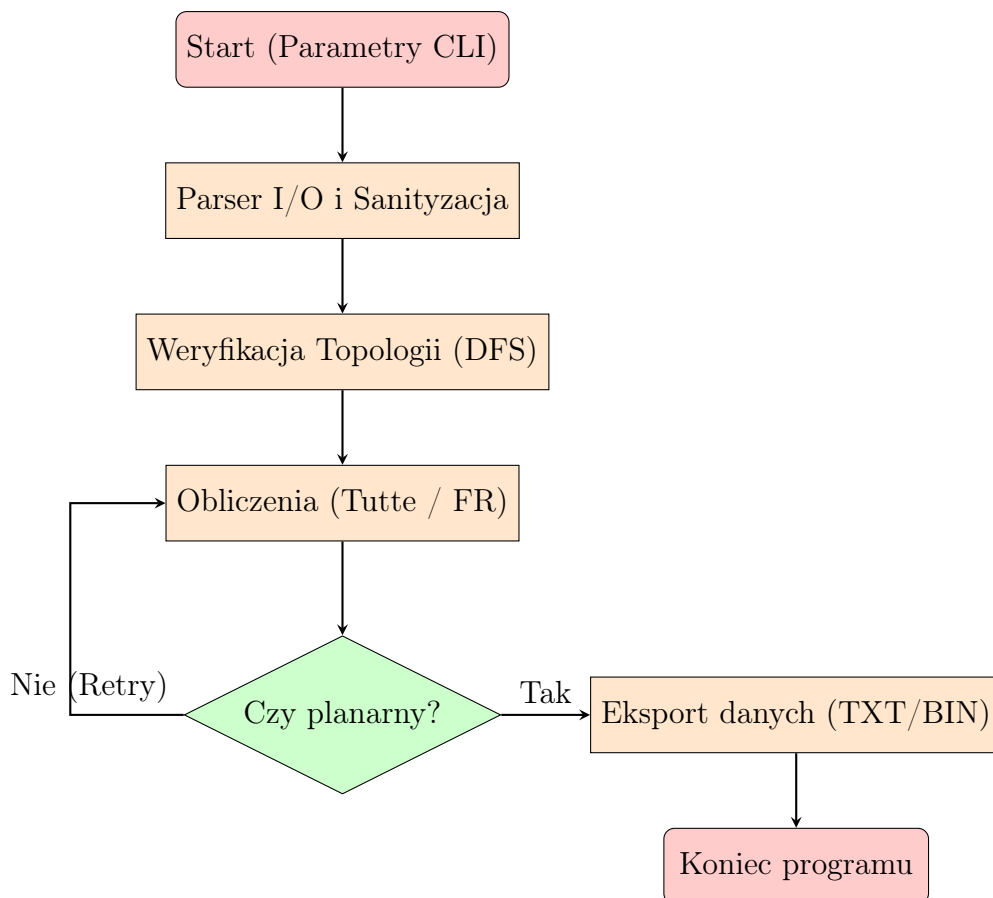
Ponieważ klasyczny model siłowy nie gwarantuje braku przecięć, w niniejszym projekcie zaimplementowano autorski moduł weryfikacji post-procesowej oparty na teście orientacji trzech punktów (CCW – Counter-Clockwise).

Wymuszanie planarności: W celu uzyskania statusu `is_layout_planar == 1`, program integruje test geometryczny z mechanizmami korygującymi:

- **Retry Logic:** Jeśli test CCW wykaże kolizję, system restartuje obliczenia z nowym rozkładem losowym (do 3000 prób), co pozwala wyjść z lokalnych minimów energetycznych.
- **Wektor grawitacji:** Dodatkowa siła dociągająca do centrum zapobiega nadmiernemu rozproszeniu wierzchołków, ułatwiając znalezienie układu bezkolizyjnego.
- **Bezpieczeństwo danych:** Mechanizm ten gwarantuje, że plik wynikowy zostanie utworzony wyłącznie dla layoutu matematycznie poprawnego.

3 Przepływ Danych i Walidacja

3.1 Schemat Przetwarzania Danych



Szczegółowa analiza potoku przetwarzania: Przepływ danych w systemie został zaprojektowany w architekturze "Pipe and Filter", gdzie każdy etap stanowi krytyczny punkt kontrolny dla stabilności aplikacji:

- **Inicjalizacja i CLI:** Program parsuje argumenty wejściowe. Błędy na tym etapie skutkują natychmiastowym przerwaniem pracy (Kod 1) i wyświetleniem instrukcji użytkownika.
- **Parser I/O i Sanityzacja:** Najbardziej rozbudowany etap, w którym następuje czyszczenie danych (zamiana przecinków na kropki), sprawdzanie niedozwolonych znaków oraz mapowanie unikalnych identyfikatorów wierzchołków na wewnętrzne indeksy tablicowe.
- **Weryfikacja Topologii:** Wykorzystanie algorytmu przeszukiwania w głąb (DFS) pozwala na odrzucenie grafów niespójnych, co jest kluczowe dla poprawności działania algorytmu Tutte'a.
- **Rdzeń Obliczeniowy:** Realizacja wybranego algorytmu layoutu. W przypadku metody Fruchtermanna-Reingolda, proces ten jest stochastyczny i zintegrowany z pętlą sprawdzającą przecięcia krawędzi.

- **Pętla Optymalizacyjna (Retry Logic):** System implementuje mechanizm automatycznego restartu obliczeń z nowymi parametrami startowymi. Jeśli test CCW (Counter-Clockwise) wykaże obecność przecięć, program wykonuje do 3000 prób w celu znalezienia konfiguracji planarnej.
- **Eksport i Utrwalanie Danych:** Po uzyskaniu poprawnego layoutu, dane są konwertowane na format docelowy (tekstowy lub binarny) i zapisywane na dysku.

4 System Diagnostyki (Kody Wyjścia)

Program komunikuje się z użytkownikiem poprzez ustandaryzowane kody powrotu:

Kod	Nazwa Błędu	Szczegóły Przyczyny
1	Argument Error	Brak flag -i/-o lub nieznany format wyjściowy.
2	Read Error	Błąd alokacji RAM lub problem z plikiem.
3	Path Error	Plik wyjściowy nie istnieje pod wskazaną ścieżką.
4	Algorithm Error	Wybrano algorytm inny niż <i>tutte/fruchterman</i> .
5	Empty File	Plik wyjściowy jest pusty.
6	Size Error	Graf zbyt mały (wymagane $V \geq 3, E \geq 2$).
7	Planarity Error	Wykryto przecięcia po zakończeniu optymalizacji.
8	Connectivity Error	Graf niespójny (test DFS niezaliczony).
9	Format Error	Nieprawidłowy format danych w pliku wejściowym (format inny niż txt lub bin).
10	Structure Error	Błędne znaki (diakrytyczne), pętle własne, błąd formatu.

Tabela 1: Tabela diagnostyczna kodów powrotu.

5 Metodologia Testowania i Scenariusze Weryfikacyjne

Proces weryfikacji oprogramowania został podzielony na trzy główne obszary: testy poprawności formatu danych (I/O), testy topologiczne oraz testy stabilności numerycznej algorytmów. Łącznie opracowano bazę 15 scenariuszy testowych.

5.1 Walidacja Składniowa i Odporność modułu I/O

W ramach testów "Black-box" sprawdzono reakcję programu na błędy w plikach wejściowych. Moduł `io.c` został zweryfikowany pod kątem:

- **Zasady czystego tekstu:** Testy z użyciem polskich znaków diakrytycznych (ą, ę, ł itp.) oraz znaków specjalnych w nazwach krawędzi – program poprawnie zwraca Kod 10.
- **Autokorekta separatorów:** Weryfikacja poprawności wczytywania wag przy użyciu przecinka zamiast kropki (test funkcji *sanitize*).
- **Sanityzacja linii:** Sprawdzenie zachowania przy napotkaniu linii pustych, nadmiarowych spacji oraz linii zaczynających się od znaku #.

5.2 Testy Topologiczne (Logika Grafowa)

Weryfikacja matematycznych założeń grafu prostego i planarnego:

- **Test Spójności:** Użycie grafów rozspójnionych (dwa osobne cykle) – potwierdzono poprawne działanie algorytmu DFS i przerwanie pracy z Kodem 9.
- **Test Planarności (K_5 i $K_{3,3}$):** Wprowadzenie grafów pełnych o gęstości przekraczającej warunek Eulera ($E > 3V - 6$). Program skutecznie blokuje zapis layoutu (Kod 8).
- **Test Rozmiaru Minimalnego:** Próba przetworzenia grafów o liczbie węzłów $V < 3$ – zwrócenie Kodu 6.

5.3 Zestawienie Scenariuszy Testowych

W poniższej tabeli przedstawiono kluczowe przypadki testowe wykorzystane podczas prac rozwojowych:

Plik Testowy	Opis Scenariusza	Oczekiwany Wynik
poprawny.txt	Graf cykliczny C_4 , poprawne dane.	Kod 0 (Sukces)
err_diakrytyczne.txt	Nazwy krawędzi zawierające znaki: "krawędź_1".	Kod 10 (Format Error)
err_niespojny.txt	Dwa niepołączone trójkąty.	Kod 9 (Connectivity)
err_pusty.txt	Plik o rozmiarze 0 bajtów.	Kod 5 (Empty File)
err_gestosc.txt	Graf pełny K_5 (10 krawędzi, 5 węzłów).	Kod 8 (Planarity)
test_przecinki.txt	Wagi zapisane w formacie "1,50".	Kod 0 (Autokorekta)
err_petla.txt	Krawędź łącząca węzeł z samym sobą (U=V).	Kod 12 (Self-loop)

Tabela 2: Wybrane przypadki testowe wykorzystane w procesie QA.

5.4 Wnioski z procesu testowania

Przeprowadzone testy wykazały, że program jest odporny na błędy typu "Human Error" przy tworzeniu plików tekstowych. Największą skuteczność wykazuje kaskadowy system walidacji, który odrzuca błędy strukturalne przed uruchomieniem kosztownych obliczeń algorytmicznych. Implementacja testu orientacji CCW w post-processingu daje 100% pewność, że pliki wyjściowe zawierają wyłącznie layouty poprawne geometrycznie.

6 Podział prac w zespole

Projekt został podzielony tak, aby każda z osób mogła pracować nad swoimi modułami niezależnie, a następnie połączyć je w całość. Poniżej przedstawiamy faktyczny podział zadań przy realizacji programu i dokumentacji.

6.1 Tabela zadań

Zadanie / Pliki	Zakres wykonanych prac	Osoba
graph.h	Opracowanie wspólnych struktur danych.	Aleksandra, Nella
fruchterman.c/h	Pełna implementacja algorytmu siłowego i fizyki.	Aleksandra
tutte.c/h	Pełna implementacja algorytmu Tutte’a.	Nella
main.c i io.c/h	Inicjalizacja plików, obsługa <code>getopt</code> i <code>help</code> .	Nella
main.c i io.c/h	Rozbudowa o walidację danych, parser, obsługę błędów i zapis BIN.	Aleksandra
Makefile, README	Przygotowanie skryptu kompilacji i instrukcji.	Nella
Bugfixing	Naprawa błędów logicznych, sprawdzanie planarności i spójności.	Aleksandra
Dokumentacja	Przygotowanie treści, opisów i skład w systemie LaTeX.	Aleksandra, Nella

Tabela 3: Zestawienie prac wykonanych przez członków zespołu.

6.2 Opis wykonanych prac

Aleksandra Rybałko Aleksandra skupiła się na części obliczeniowej i poprawności danych. Przygotowała plik nagłówkowy `graph.h` oraz w pełni zaimplementowała algorytm Fruchtermana. W modułach `main` i `io` odpowiadała za kluczowe elementy: parser (z obsługą zamiany przecinków na kropki), walidację wejścia oraz zapis wyników do plików tekstowych i binarnych. Zajęła się także końcową integracją kodu, usuwaniem błędów oraz dodaniem warunków sprawdzających planarność i spójność grafu. Przygotowała również zestaw plików testowych do weryfikacji działania programu.

Nella Karczewicz Nella stworzyła podstawy projektu: zaimplementowała algorytm Tutte’a oraz przygotowała szkielety plików `main.c` i `io.c`. Odpowiadała za techniczną stronę interfejsu CLI, wprowadzając obsługę flag przez `getopt` oraz opcję pomocy. Jest autorką `Makefile` i pliku `README`. Brała główny udział w tworzeniu dokumentacji — zwłaszcza części implementacyjnej i końcowej — oraz przygotowała zestaw plików testowych do sprawdzania poprawności działania programu.

Zadania wspólne Wspólnie prowadziłyśmy integrację modułów oraz projektowanie elastycznej struktury danych dla wierzchołków i krawędzi. Opracowałyśmy system zapisu wyników do plików tekstowych i binarnych oraz przeprowadziłyśmy debugowanie i optymalizację kodu, eliminując błędy logiczne i problemy z pamięcią.

Równolegle pracowałyśmy nad dokumentacją, przygotowując opisy funkcjonalne i implementacyjne. Stworzyłyśmy też kompletną bazę scenariuszy testowych — poprawnościowych, odpornościowych i I/O — zapewniając stabilność działania programu.

7 Podsumowanie

Projekt dostarcza narzędzie do wizualizacji grafów planarnych, łączące podejście deterministyczne z heurystycznym. Największym wyzwaniem była implementacja odpornego parsera, który oprócz czyszczenia danych i ignorowania komentarzy wykonuje autokorektę separatorów dziesiętnych oraz rygorystyczną weryfikację typów, dzięki czemu poprawnie przetwarza także nietypowe pliki wejściowe. Istotnym elementem było również zapewnienie planarności layoutów: algorytm Tutte’a gwarantuje ją matematycznie, natomiast model Fruchtermanna-Reingolda został rozszerzony o mechanizm „strażnika planarności”, który po każdej symulacji wykrywa kolizje krawędzi i w razie potrzeby restartuje obliczenia. Testy wykazały, że Tutte działa bardzo szybko i stabilnie, a FR — mimo większej złożoności — oferuje wyższą jakość wizualną i równomierny rozkład wierzchołków. W efekcie powstał system odporny na błędy i zapewniający przejrzyste, czytelne wyniki.